ISTANBUL UNIVERSITY-CERRAHPAŞA

Department of Computer Engineering

FILE ORGANIZATION PROJECT

Password Dataset — Storage, Search & Performance Analysis

Semester	Spring 2026
Programming Language	C (C99 Standard)
Compiler	GCC — Apple Silicon / macOS
Build System	GNU Make (Makefile)
Dataset	German_common-password-list-top-100000 (SecLists)
External Libraries	zlib, OpenSSL (libcrypto)
Evaluated Methods	Raw (Plain Text) | Hashed (SHA-256) | Compressed (zlib)

Student Information
Name: Serdar Ulaş Budak
Student No: 1306230038
Email: serdarulasbudak@ogr.iuc.edu.tr

Course Instructors
Assoc. Prof. Dr. Derya Yıltaş Kaplan
Research Asst. Melike Başer

June 2026

1. Introduction

This project examines how storing the same password dataset in three different formats — **plain text (raw)**, **SHA-256 hash**, and **zlib-compressed binary** — affects disk usage, search speed, CPU load, and memory consumption. The dataset used is **German_common-password-list-top-100000**, sourced from Daniel Miessler's SecLists repository.

Efficient storage and lookup of large password lists is a critical requirement in password security. Hash-based storage prevents plaintext exposure, while compression aims to reduce disk footprint — at the cost of a decompression overhead at query time. This study quantifies those trade-offs with measured data.

The entire project is implemented in C (C99). All inter-module dependencies are resolved through a single central header `sdk.h`, and the build process is fully automated with GNU Make.

2. Tools and Technologies

2.1 Language and Compiler

The project is written in C99 and compiled with GCC on an Apple Silicon (ARM-based) macOS system. Build automation is handled by GNU Make, which ensures incremental compilation — only changed source files are recompiled.

2.2 Central Header — sdk.h

All modules include a single `sdk.h`, which provides:

- An include guard (`#ifndef SDK_H`) to prevent multiple-definition errors
- Centralized standard-library includes (`stdio.h`, `stdlib.h`, `string.h`, `stdint.h`, `openssl/evp.h`)
- Core function prototypes and a chain of per-module headers (`compression.h`, `hash.h`, `preprocess.h`, `search.h`, `storage.h`, `utils.h`)

Any `.c` file therefore needs only `#include "sdk.h"`.

2.3 Standard Libraries

Library / Header	Purpose
stdio.h	File I/O: fopen, fclose, fread, fwrite, fseek, ftell, fprintf, fgets
stdlib.h	Memory management: malloc, calloc, free; numeric conversions
string.h	String operations: strcmp, strlen, strcspn, memcmp
stdint.h	Portable integer types: uint32_t, uint64_t
time.h	Timing: clock(), CLOCKS_PER_SEC
zlib	DEFLATE compression/decompression: compress(), uncompress(),

Library / Header	Purpose
	compressBound()
OpenSSL (EVP)	SHA-256 digest: EVP_DigestInit_ex(), EVP_DigestUpdate(), EVP_DigestFinal_ex()
mach/mach.h	macOS: resident RAM measurement via task_info() / MACH_TASK_BASIC_INFO
sys/resource.h	POSIX: CPU time via getrusage() (ru_utime + ru_stime)

2.4 zlib

zlib implements the RFC 1951 DEFLATE standard. The functions used are `compressBound()` to pre-allocate a safe output buffer, `compress()` to deflate data, and `uncompress()` to restore the original. Return codes `Z_OK` and `Z_MEM_ERROR` are checked after every call.

2.5 OpenSSL — libcrypto (EVP API)

The EVP (Envelope) interface provides a uniform API for cryptographic digests. SHA-256 is selected via `EVP_sha256()`; switching to SHA-512 would require changing only that one call. The context lifecycle — `EVP_MD_CTX_new()`, `EVP_DigestInit_ex()`, `EVP_DigestUpdate()`, `EVP_DigestFinal_ex()`, `EVP_MD_CTX_free()` — is followed strictly to prevent memory leaks.

2.6 Platform-Specific Measurement

CPU and RAM measurement functions in `utils.c` are selected at compile time via `#ifdef` directives:

- `mach/mach.h` + `task_info()` for resident RAM on macOS
- `sys/resource.h` + `getrusage()` for CPU time on POSIX systems
- `windows.h` / `psapi.h` on Windows — no manual configuration required

3. Project Design and Architecture

3.1 Folder Structure

The project follows a strict separation of concerns across the following directories:

```
Project/
├── RawData/        ← Original password file (never modified)
├── Processed/      ← Cleaned, deduplicated data (Processed.txt)
├── Storage/
│   ├── Raw/        ← Plain-text copy (baseline)
│   ├── Hashed/     ← SHA-256 hex lines (Hashed.txt)
│   └── Compressed/ ← zlib binary file (Compressed_Data.bin)
├── Code/           ← All C source files
├── Results/        ← Performance output files
└── Report/         ← This PDF
```

3.2 Data Flow

Step	Description
1 — Raw Read	German_common-password-list-top-100000 is kept in RawData/ untouched.
2 — Preprocessing	preprocess.c reads line-by-line, filters, deduplicates, and writes Processed.txt.
3 — Raw Storage	copy_file() copies Processed.txt to Storage/Raw/ in binary mode.
4 — Hash Storage	main_hash() computes SHA-256 for each line and writes the 64-char hex string to Hashed.txt.
5 — Compressed Storage	main_compress() deflates each line with per-record metadata and writes Compressed_Data.bin.
6 — Search	search.c performs linear search over all three formats; timing, CPU, and RAM are recorded.

3.3 Module Architecture

File	Role	Key Functions
main.c	Orchestration; drives all phases	main(), main_process(), main_hash(), main_compress(), main_decompress()
preprocess.c	Reading, filtering, deduplication	satiroku(), process(), tekrar_kontrol(), temizle_newline()
storage.c	Writing to all three formats	dosya_yaz(), dosyaya_sikistirarak_yaz()
search.c	Linear search over all three formats	rawData_search(), hashed_data_search(), search_compressed_file()
compression.c	zlib compress / decompress logic	sikistir(), veri_ac(), dosyadan_oku_ve_ac()
hash.c	SHA-256 digest via OpenSSL EVP	sha256_hash()
utils.c	Timing, disk size, CPU/RAM measurement	start_timer(), stop_timer(), get_file_size(), copy_file(), get_cpu_usage(), get_ram_usage()
sdk.h	Central header: types, prototypes, chain	include guard + prototypes + module headers

4. Data Preprocessing

4.1 Reading Raw Data — satiroku()

`satiroku()` reads up to 256 characters per line using `fgets()`. After each read, `temizle_newline()` replaces trailing \n (Unix) and \r (Windows CRLF) with '\0'. The function returns NULL at EOF, terminating the calling loop. Each invocation allocates 256 bytes via `malloc()`; the caller is responsible for calling `free()` to prevent memory leaks.

4.2 Filtering Rules — process()

Each line passes through four sequential filters:

- **Character elimination:** Only lowercase ASCII letters (a–z) and digits (0–9) are retained; uppercase, locale-specific characters (ä, ö, ü, etc.), and symbols are removed.
- **Length check:** The cleaned string must be at least 7 characters long.
- **Mixed-character requirement:** The entry must contain at least one digit AND at least one letter; purely numeric or purely alphabetic strings are discarded.
- **NULL safety:** If the input is NULL, the function returns immediately.

4.3 Deduplication — tekrar_kontrol()

Before writing to `Processed.txt`, each candidate is checked against the file's existing content by seeking to `SEEK_SET` and comparing every line with `strcmp()`. The function returns `0` if a duplicate is found and `1` if the entry is unique. This O(n^2) approach is acceptable at the project scale.

4.4 SHA-256 Digest Generation — sha256_hash()

The `sha256_hash()` function in `hash.c` uses the OpenSSL EVP API to produce a 256-bit (32-byte) digest.

- The EVP API is chosen over the deprecated direct SHA functions to keep algorithm changes to a single line
- `EVP_DigestUpdate()` supports incremental feeding
- `EVP_MD_CTX_free()` is always called to release the context

The 32-byte raw digest is converted to a 64-character hex string via `sprintf("%02x")` and written to file.

5. Storage Methods

5.1 Raw Storage — Plain Text

`Processed.txt` is copied to `Storage/Raw/RawData.txt` in binary mode (`rb` / `wb`) using `copy_file()`. Each record is stored as-is with no additional transformation. This format serves as the baseline for all comparisons.

> 📁 **Measured file size:** 619,091 bytes (~605 KB)

5.2 Hashed Storage — SHA-256

The original password text is never stored. Instead, the 64-character hex SHA-256 digest of each line is written to `Hashed.txt`. Because SHA-256 is a one-way function, the plaintext cannot be recovered in practice.

> ⚠️ **Why is the file larger?** Raw lines average ~10 characters, whereas every SHA-256 hex output is a fixed 64+1 bytes. Hashed.txt reaches 4,046,848 bytes (~3.86 MB) — approximately 6.54× the raw file.

5.3 Compressed Storage — zlib

Each line is compressed individually and written to `Compressed_Data.bin`. The write function `dosyaya_sikistirarak_yaz()` performs the following steps for each record:

- Compute `original_size = strlen(data) + 1` (including null terminator)
- Call `compressBound()` to determine the maximum safe output buffer size; allocate with `malloc()`
- Call `compress()` to deflate the data using DEFLATE
- Write to file: (a) `original_size` as 8 bytes, (b) `compressed_size` as 8 bytes, (c) the compressed bytes
- Call `free()` on the allocated buffer

> ℹ️ **Note:** For short strings, the 16-byte metadata overhead per record exceeds the space DEFLATE saves. Measured size: 2,110,341 bytes (~2.01 MB); compression ratio: 3.00.

6. Search Algorithms and Performance Measurement

6.1 Search Functions

rawData_search(): Seeks to file start (`SEEK_SET`) and reads line-by-line via `satiroku()`. Each line is compared with `strcmp()`. On a match, the byte offset is returned via `ftell()`; otherwise `-1` is returned.

hashed_data_search(): The query password is first hashed with `sha256_hash()` and formatted as a 64-character hex string. The file is then scanned line-by-line. The SHA-256 computation is performed only once per query call; the dominant cost is the linear file scan over the 3.86 MB file.

search_compressed_file(): Each record is decompressed individually in memory via `dosyadan_oku_ve_ac()` and compared with `strcmp()`. In-memory DEFLATE decompression is extremely fast (23.63 ms for the entire file), producing the shortest search times despite the extra decompression step.

6.2 Timing Measurement

`start_timer()` records the current `clock()` value in a static variable. When `stop_timer()` is called, the difference is divided by `CLOCKS_PER_SEC` and multiplied by 1000 to yield milliseconds. All 200 queries are executed sequentially in a single for-loop.

6.3 CPU Usage Measurement

`get_cpu_usage()` computes the ratio of CPU time to wall-clock time between two consecutive calls. On macOS it uses `getrusage()` and `gettimeofday()`. On Windows it uses `GetProcessTimes()` and `GetSystemTime()`. Formula: `cpu_diff / wall_diff × 100`.

6.4 RAM Usage Measurement

`get_ram_usage()` reads the process's resident set size: on macOS via `task_info()`; on Linux via `/proc/self/statm`; on Windows via `K32GetProcessMemoryInfo()` WorkingSetSize. All values are returned in MB.

6.5 Disk Size Measurement

`get_file_size()` opens the file in `"rb"` mode, seeks to `SEEK_END` with `fseek()`, and returns the byte position from `ftell()`.

7. Results, Evaluation & Conclusion

7.1 Pipeline Execution Status

Phase	Name	Description	Status
Phase 1	Data Preprocessing	Raw file read line-by-line; filtered (≥7 chars, mixed alpha+digit); duplicates removed → Processed.txt	✓ DONE
Phase 2	Hashing (SHA-256)	Each processed line hashed with OpenSSL EVP SHA-256 → 64-char hex → Hashed.txt	✓ DONE
Phase 3	Compression (zlib)	Each line DEFLATE-compressed with 16-byte metadata prefix → Compressed_Data.bin	✓ DONE
Phase 4	Decompression	Compressed_Data.bin read back, each record decompressed → Decompressed.txt (integrity check)	✓ DONE
Phase 5	Search Operations	200-query benchmark executed over Raw, Hashed and Compressed formats; timing, CPU, RAM recorded	✓ DONE

7.2 Disk Usage

Format	File	Size (bytes)	Size (KB)	Size (MB)	vs Raw
Raw	RawData.txt	619,091	604.97	0.59	1.00× (base)
Hashed	Hashed.txt	4,046,848	3,952.00	3.86	6.54×

Format	File	Size (bytes)	Size (KB)	Size (MB)	vs Raw
Compressed	Compressed_Data.bin	2,110,341	2,061.07	2.01	3.41×

Table 2 — Exact disk sizes. Compression ratio (Compressed/Raw) = 3.00 because per-record 16-byte metadata overhead exceeds DEFLATE gain on short strings.

7.3 Search Performance

Comparison	Time Difference	Speedup Factor	% Improvement
Compressed vs Raw	6,062.93 ms	2.51×	60.15%
Compressed vs Hashed	6,633.13 ms	2.65×	62.30%
Raw vs Hashed	571.20 ms	1.06×	5.67%

💡 **Key Insight:** Total decompression time for the entire Compressed_Data.bin: 23.63 ms — only 0.59% of the 4,013 ms total search time. In-memory DEFLATE is negligible; the bottleneck is linear file traversal, not the decompression step itself.

7.4 CPU & RAM Usage

Metric	Raw	Hashed	Compressed	Interpretation
CPU Usage (avg.)	99.99%	99.99%	99.97%	Single-threaded sync I/O; CPU never idle
RAM Usage (avg.)	4.16 MB	4.16 MB	4.16 MB	Line-by-line reading; file size irrelevant
RAM Std. Deviation	0 MB	0 MB	0 MB	Completely stable; no bulk memory allocation

7.5 Complete Performance Summary

Metric	Raw (Plain Text)	Hashed (SHA-256)	Compressed (zlib)
Disk Size (bytes)	619,091	4,046,848	2,110,341
Disk Size (KB)	604.97	3,952.00	2,061.07
Disk Size (MB)	0.59	3.86	2.01
Size vs Raw	1.00× (base)	6.54×	3.41×

Metric	Raw (Plain Text)	Hashed (SHA-256)	Compressed (zlib)
Total Search Time (200q)	10,075.31 ms	10,646.51 ms	4,013.38 ms
Avg. Search / Query	50.38 ms	53.23 ms	20.07 ms
CPU Usage (avg.)	99.99%	99.99%	99.97%
RAM Usage (avg.)	4.16 MB	4.16 MB	4.16 MB
Compression Ratio	—	—	3.00
Decompression Time	—	—	23.63 ms
Decompression % of Search	—	—	0.59%
Queries Found	60 / 200	60 / 200	60 / 200
Hit Rate	30.0%	30.0%	30.0%
Security Level	Low	High	Medium
Search Speed Rank	2nd	3rd (slow)	1st (fast)
Disk Efficiency Rank	1st (best)	3rd (worst)	2nd

7.6 Search Results — Found Passwords (Sample)

Metric	Raw	Hashed	Compressed
Total Queries	200	200	200
Found	60	60	60
Not Found	140	140	140
Hit Rate	30.0%	30.0%	30.0%

Password	Raw Offset (B)	Hashed Offset (B)	Compressed Offset (B)
123456a	364	2,535	1,300
qwerty123	372	2,600	1,332
florian1	382	2,665	1,366
1qaz2wsx	391	2,730	1,399
killer123	409	2,860	1,465
alexander1	428	2,990	1,532
passwort123	447	3,120	1,599
test123	490	3,380	1,738

Password	Raw Offset (B)	Hashed Offset (B)	Compressed Offset (B)
porsche911	498	3,445	1,770
hallo1234	538	3,705	1,906
… (early-file entries above, late-file entries below)	…	…	…
patrick2010	544,755	3,574,285	1,861,910
zahlen123	544,822	3,574,740	1,862,145
husqvarna125	545,090	3,576,495	1,863,061

Table 7 — Sample of found passwords with byte offsets in each format (16 of 60 shown).

7.7 Best-in-Class Comparison

Criterion	Best Method	Reason
Disk Usage	Raw	No per-record metadata; smallest footprint (604 KB)
Search Speed	Compressed	In-memory DEFLATE is negligible (0.59%); smaller I/O file
Security	Hashed	SHA-256 is one-way; plaintext never stored or recoverable
Implementation Ease	Raw	No external libraries; single strcmp() comparison
Scalability	Compressed	Compression ratio drops below 1.0 for longer record sets
Portability	Hashed	Fixed 64-byte output is platform-independent
RAM Efficiency	All Equal	All formats: 4.16 MB — line-by-line reading prevents bulk load

7.8 Recommended Use-Case Scenarios

Use Case	Recommended	Rationale
High-frequency query database	Compressed	60% faster search than Raw/Hashed; handles millions of daily queries
Password security system	Hashed	Plaintext never stored; breach-resistant even if file is leaked
Archive / backup storage	Compressed	Smaller binary vs Hashed; 23.63 ms decompress is acceptable
Low-dependency prototype	Raw	No external libs needed; direct strcmp(); ideal for dev/testing

7.9 Overview & Interpretation

This benchmark evaluated three fundamentally different strategies for storing and searching the same password dataset. The experiment ran 200 search queries against each format on an Apple Silicon macOS machine, measuring search time, disk footprint, CPU load, and memory consumption. The results reveal clear and measurable trade-offs that have direct implications for real-world system design.

Search Speed — Compressed Wins Decisively

The most counterintuitive result of this study is that the Compressed format produced the fastest search times — **4,013 ms** total versus **10,075 ms** for Raw and **10,647 ms** for Hashed. This represents a 2.51× speedup over Raw and a 2.65× speedup over Hashed. The key insight is that in-memory DEFLATE decompression consumed only 23.63 ms — a mere 0.59% of total search time. The actual bottleneck in all three methods is disk I/O: the Compressed binary file (2.01 MB) is smaller than the Hashed text file (3.86 MB), so fewer bytes traverse the I/O bus per query.

Disk Usage — Raw Is the Clear Winner, Hashed Is Surprisingly Costly

The Raw format occupies only **604.97 KB** — by far the most compact representation. The Hashed format inflates to **3.86 MB** — 6.54× the raw size — because every SHA-256 digest is a fixed 65 bytes per record while original entries average ~10 characters. The Compressed format reaches **2.01 MB** — smaller than Hashed but still 3.41× larger than Raw due to 16-byte per-record metadata overhead.

CPU and RAM — No Meaningful Difference

All three methods sustained approximately 100% CPU utilization throughout the benchmark. This is the expected behavior of single-threaded, synchronous I/O-bound workloads. RAM consumption was identical for all methods at **4.16 MB**, thanks to the line-by-line reading design of `satiroku()`. The same code can process a 10 GB password list without requiring more than a few MB of RAM.

Security — Only Hashed Provides Real Protection

From a security standpoint, only the Hashed format genuinely protects user data. Raw storage exposes every password as plaintext. Compressed storage is no better — decompressing the binary file is trivial. The Hashed format is qualitatively different: SHA-256 is a one-way cryptographic function, making it the only acceptable choice in any production security context.

Search Accuracy — Why Only 30% Were Found

Of the 200 queried passwords, only 60 (30%) were found across all three formats. This is consistent and expected: the 70% miss rate reflects the aggressive filtering applied during preprocessing. The `process()` function discards any entry shorter than 7 characters, containing no digits, no letters, or characters outside the lowercase ASCII alphanumeric set.

Practical Recommendations

No single method dominates across all criteria. An optimal production architecture might store Hashed as the authoritative secure record, maintain a Raw or Compressed index for fast lookup, and use Compressed for long-term archival.

Limitations and Future Work

- This experiment used linear O(n) search over flat files — a B-tree or hash table would reduce average lookup to O(log n) or O(1)
- A page cache or memory-mapped file (`mmap`) would reduce repeated disk I/O significantly
- The SHA-256 digest is recomputed on every hashed search call; pre-computing and caching the query hash would eliminate that overhead
- The compression results are specific to this dataset's short-string characteristics — longer records would likely invert the compression ratio below 1.0

8. References

[1] Daniel Miessler, SecLists — German_common-password-list-top-100000. https://github.com/danielmiessler/SecLists/blob/master/Passwords/Common-Credentials/german_common-password-list-top-100000.txt

[2] zlib Official Documentation — compress(), compressBound(), uncompress() API. https://zlib.net/manual.html

[3] RFC 1951 — DEFLATE Compressed Data Format Specification. https://www.rfc-editor.org/rfc/rfc1951

[4] OpenSSL EVP Digest API. https://www.openssl.org/docs/man3.0/man3/EVP_DigestInit.html

[5] POSIX getrusage(). https://pubs.opengroup.org/onlinepubs/9699919799/functions/getrusage.html

[6] Apple Developer — task_info() / MACH_TASK_BASIC_INFO. https://developer.apple.com/documentation/kernel/1537934-task_info

[7] Linux Programmer's Manual — proc(5) / /proc/self/statm.

[8] Microsoft Docs — K32GetProcessMemoryInfo(). https://docs.microsoft.com/en-us/windows/win32/api/psapi/

[9] GNU Make Manual. https://www.gnu.org/software/make/manual/make.html

[10] Brian W. Kernighan, Dennis M. Ritchie — The C Programming Language, 2nd Ed., Prentice Hall, 1988.